

ShopSphere Efficiency Hackathon

Time & Space Complexity · Retail E-Commerce · Problem Statements

DEVELOPED BY TALENCIAGLOBAL

About This Hackathon

ShopSphere is a fast-growing retail e-commerce platform. Across five levels you will solve real problems from its catalog, checkout, merchandising and finance systems — and you will be judged not only on correctness but on the running time and memory your solution uses.

Each level states a complexity target. A solution that returns the right answer but misses the target is only a partial solution: the goal is to reason about Big-O, Big-Theta and Big-Omega, weigh time against space, and choose the right algorithm. The levels build on one another and grow harder. No starter code is provided — design your own.

Correctness

Your solution must produce the right answer for all inputs.

Complexity

Meet the stated Big-O time and space targets for full credit.

Design

No starter code — architect your own solution from scratch.

Levels

1	Level 1 Cart Total via Hash-Map Lookup EASY
2	Level 2 Top-K Best Sellers MEDIUM
3	Level 3 Shortest Flash-Sale Window MEDIUM-HARD
4	Level 4 Promo Segments with an Exact Total HARD
5	Level 5 Gift-Card Bundle Builder EXPERT

EASY

Level 1 — Cart Total via Hash-Map Lookup

Constant vs Linear Time — $O(1)$ and $O(n)$

Challenge

ShopSphere's checkout must total a customer's cart against the product catalog. With a large catalog and large carts, the lookup strategy decides whether checkout feels instant or sluggish. Price the cart as efficiently as possible.

Learning Objectives

- What Big-O actually measures
- Constant time $O(1)$ versus linear time $O(n)$
- Hash maps for $O(1)$ average lookup
- Turning an $O(k \times m)$ approach into $O(m + k)$

Level 1 — Problem Statement

Given

- A catalog of m products, each with an id and a unit price
- A cart of k lines, each an id and a quantity

Constraints

- $1 \leq m \leq 100,000$
- $1 \leq k \leq 100,000$
- $0 \leq \text{price} \leq 1,000,000$
- $1 \leq \text{quantity} \leq 1,000$
- Every cart id exists in the catalog

Input Format

Line 1: m . Next m lines: id price. Next line: k . Next k lines: id quantity.

Output Format

Print one line → Cart Total: <sum of price $\times$ quantity>

✓ **Complexity Target:** $O(m + k)$ time, $O(m)$ space

Sample Input

```
4
SKU1 100
SKU2 250
SKU3 75
SKU4 500
3
SKU2 2
SKU4 1
SKU1 3
```

Sample Output

```
Cart Total: 1300
```

Extension Challenge

- Apply per-product discounts and a cart-level coupon.
- Support cart ids that may be missing from the catalog.

MEDIUM

Level 2 — Top-K Best Sellers

Linearithmic Time — $O(n \log n)$ vs $O(n \log k)$

Challenge

The merchandising dashboard shows the K best-selling products. Sorting the entire catalog works, but when K is small and the catalog is huge, a bounded heap does far less work. Return the top K, ranked by sales with ties broken by product id.

Learning Objectives

- Why comparison sorting is $\Theta(n \log n)$
- The $\Omega(n \log n)$ lower bound for comparison sorts
- A bounded heap or ordered set for $O(n \log k)$
- Choosing an algorithm by the size of k relative to n

Level 2 — Problem Statement

Given

- n products, each with an id and a sales count
- An integer k

Constraints

- $1 \leq k \leq n \leq 200,000$
- $0 \leq \text{sales} \leq 1,000,000,000$
- Rank by sales descending, breaking ties by id ascending

Input Format

Line 1: n . Next n lines: id sales. Next line: k .

Output Format

Print k lines, best first $\rightarrow$ `<id> <sales>`

✓ **Complexity Target:** $O(n \log k)$ time, $O(k)$ space

Sample Input

```
5
APPLE 120
BANANA 300
CHERRY 300
DATE 50
ELDER 210
3
```

Sample Output

```
BANANA 300
CHERRY 300
ELDER 210
```

Extension Challenge

- Update the ranking in real time as new sales stream in.
- Rank by revenue (sales $\times$ price) instead of raw units sold.

A desk setup featuring a laptop on the left, a cup of coffee in the top right, a calendar with revenue data in the center, and a smartphone on the right. The calendar shows a grid of days with revenue values: \$12,450, \$8,320, and \$15,900. The smartphone displays a line graph and some statistics.

MEDIUM-HARD

Level 3 — Shortest Flash-Sale Window

Linear Time via Two Pointers — $O(n)$ vs $O(n^2)$

Challenge

Marketing wants the shortest run of consecutive days whose combined revenue reaches a flash-sale target. Checking every possible run is $O(n^2)$; a sliding window finds the answer in a single pass.

Learning Objectives

- Replacing nested loops $O(n^2)$ with a linear scan $O(n)$
- The sliding-window / two-pointer pattern
- Amortized analysis: why each element is visited at most twice
- How non-negative values make the window valid

Level 3 — Problem Statement

Given

- n daily revenue values, all non-negative
- A target revenue T

Constraints

- $1 \leq n \leq 1,000,000$
- $0 \leq \text{revenue}[i] \leq 1,000,000,000$
- $1 \leq T \leq 1,000,000,000,000,000$
- Revenues are non-negative, which the window relies on

Input Format

Line 1: n . Line 2: n revenue values. Line 3: T .

Output Format

Print one line $\rightarrow$ Shortest Window: $\langle \text{length} \rangle$ (0 if no run reaches T)

Sample Input

```
8
2 3 1 2 4 3 1 5
7
```

Sample Output

Shortest Window: 2

Extension Challenge

- Return the actual start and end days of the shortest window.
- Handle a variant that asks to reach T within at most D days.

✔ **Complexity Target:** $O(n)$ time, $O(1)$ space

HARD

Level 4 — Promo Segments with an Exact Total

Trading Space for Time — $O(n)$ time, $O(n)$ space

Challenge

Finance must count the contiguous transaction segments whose net total equals a promotional value K . Transactions include refunds (negative values), so the sliding window no longer applies. A prefix-sum hash map counts every segment in one pass, at the cost of extra memory.

Learning Objectives

- Prefix sums to express any segment total in $O(1)$
- A hash map of seen prefixes for $O(n)$ counting
- Making the space-time trade-off explicit
- Why negative values break the sliding window

Level 4 — Problem Statement

Given

- n transactions, any integer (positive sales or negative refunds)
- A target value K

Constraints

- $1 \leq n \leq 1,000,000$
- $-1,000,000,000 \leq \text{transaction}[i] \leq 1,000,000,000$
- K fits in a 64-bit integer
- Values may be negative, so a sliding window cannot be used

Input Format

Line 1: n . Line 2: n transactions. Line 3: K .

Output Format

Print one line → Matching Segments: <count>

Sample Input

```
6
3 4 7 -3 3 1
7
```

Sample Output

Matching Segments: 3

Extension Challenge

- Also report the longest qualifying segment, not just the count.
- Count segments whose total is divisible by K instead of equal to K .

✔ **Complexity Target:** $O(n)$ time, $O(n)$ space

EXPERT

Level 5 — Gift-Card Bundle Builder

Exponential vs Pseudo-Polynomial — Dynamic Programming

Challenge

A customer holds a gift card worth B . From a set of products, each with a cost and a delight score, choose a subset that maximizes total delight without exceeding the budget. Trying every subset is $O(2^n)$; dynamic programming makes it tractable.

Learning Objectives

- Recognizing the 0/1 knapsack structure
- Exponential brute force $O(2^n)$ versus DP $O(n \times B)$
- Pseudo-polynomial complexity (it depends on the value B)
- Collapsing a 2-D DP table to 1-D for $O(B)$ space

Level 5 — Problem Statement

Given

- n products, each with a cost and a delight value
- A budget B

Constraints

- $1 \leq n \leq 1,000$
- $1 \leq B \leq 100,000$
- $0 \leq \text{cost}[i] \leq B$
- $0 \leq \text{value}[i] \leq 1,000,000,000$

Input Format

Line 1: n B . Next n lines: cost value.

Output Format

Print one line → Max Delight: <maximum total value within budget>

✔ **Complexity Target:** $O(n \times B)$ time, $O(B)$ space

Sample Input

```
4 10
2 3
3 4
4 5
5 6
```

Sample Output

Max Delight: 13

Extension Challenge

- Reconstruct which products were chosen, not just the best value.
- Add a cap on the number of items allowed in the bundle.